**Université de Haute-Alsace**
**ENSISA**

**ENSISA IR 3ème année**
**Année 2026/2027**

# Programmation Rust

TP3 : Types, erreurs et tests

---

## Objectifs

Définir des énumérations pour exprimer plusieurs états possibles.

Exploiter `match` pour traiter de façon exhaustive les variantes d'une énumération.

Utiliser `Option<Player>` lorsqu'un vainqueur peut être absent.

Utiliser `Result<T, MoveError>` lorsqu'une opération peut échouer.

Séparer les règles du jeu de la gestion des entrées utilisateur.

Voir un exemple de test.

---

## Instructions

Lisez bien l'énoncé, il contient beaucoup d'informations et chaque mot est utile !

Aidez-vous de la documentation que je vous indiquerai. Lorsqu'il utilise une nouvelle librairie, un développeur passe beaucoup de temps dans la documentation pour comprendre son fonctionnement.

Codez proprement avec des noms de variables et de fonctions/méthodes bien choisis. Ajoutez des commentaires utiles à votre code afin de pouvoir encore le comprendre dans quelques jours.

Respectez le coding style de Rust[1]. `cargo fmt` peut déjà corriger certaines choses.

Simple is beautiful. N'essayez pas de coder compliqué, restez simple, cela sera plus efficace et moins sujet aux erreurs, surtout en Rust.

Réfléchissez avant de coder, prenez un peu de temps pour dessiner/écrire vos idées sur une feuille. Le temps de la réflexion que vous prendrez, vous économisera énormément de temps de debug.

Utiliser `cargo clippy` pour vérifier votre code.

---

## Contexte

Dans les TP précédents, vous avez développé un morpion jouable dans un terminal, puis organisé le programme autour des structures Board et TicTacToe et de leurs méthodes.

Le programme fonctionne, mais sa première version s'appuie sur des conventions implicites : 0 signifie « case vide », 1 et 2 représentent les joueurs, et une fonction de victoire renvoie seulement un booléen. Ces choix fonctionnent sur un petit programme, mais ils deviennent vite fragiles quand le code évolue.

1. Style guide de Rust : `https://doc.rust-lang.org/style-guide/`

**Votre travail :** conserver le morpion en ligne de commande tout en représentant explicitement les états du jeu et les erreurs grâce aux types de Rust. Vous utiliserez le fichier **`main.rs`**, qui contient déjà l'architecture et la boucle d'interaction, mais dont **neuf blocs `todo!()` sont à compléter**.

À la fin du TP, le programme devra accepter les coups `A1` à `C3`, refuser les coups incorrects sans changer de joueur, annoncer le vainqueur ou le match nul, puis proposer de rejouer.

# 1 Exploration fichier

Créez un nouveau projet `cargo` et mettez le fichier `main.rs` dedans.

Repérez les structures `Board`, `TicTacToe` et les types `Player`, `Cell`, `GameState` et `MoveError`.

Compilez et exécutez avec `cargo`.

Attention : le fichier de départ compile, mais les appels aux fonctions contenant `todo!()` provoquent volontairement une erreur à l'exécution. Le but du TP est de remplacer tous ces `todo!()` par votre code.

Pour compiler et exécuter les tests présents en bas du fichier : `cargo test`

Les tests concernés échoueront tant que les méthodes ne sont pas correctement implémentées.

# 2 Exercice 1

Le fichier de départ contient déjà les énumérations suivantes :

```rust
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
enum Player { X, O }

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
enum Cell { Empty, Occupied(Player) }
```

**À faire :**

— Complétez `Player::symbol(self) -> char` avec `match` pour obtenir `'X'` ou `'O'`.
— Complétez `Player::other(self) -> Player` pour obtenir le joueur suivant.
— Complétez `Cell::symbol(self) -> char` : une case vide affiche un espace, une case occupée affiche le symbole de son joueur.

**Questions :** Pourquoi `Cell::Occupied(Player)` est-il plus expressif que `1` ou `2` ? À quoi servent les dérivations `Copy` et `PartialEq` dans ce programme ?

**Validation :** le test `player_and_cell_symbols` réussit.

# 3 Exercice 2 : `Result`

Écrivez le corps de :

```rust
fn parse_move(input: &str) -> Result<(usize, usize), MoveError>
```

La fonction reçoit la chaîne tapée par l'utilisateur et doit renvoyer :

— `Ok((ligne, colonne))` si le coup est compris entre `A1` et `C3` ;
— `Err(MoveError::InvalidPosition)` dans tous les autres cas.

Les indices vont de `0` à `2`. Ainsi, `A1` correspond à `(0, 0)` et `C3` à `(2, 2)`. Les espaces autour de la saisie doivent être ignorés et les lettres minuscules acceptées (`b2` équivaut à `B2`).

**Conseils :** `trim()` supprime les espaces autour de la saisie, `to_ascii_uppercase()` normalise les lettres et `match` permet de convertir les lettres et les chiffres en indices.

**Validation :** les tests `parse_move_accepts_valid_input` et `parse_move_rejects_invalid_input` réussissent. Testez en particulier `A4`, `D1`, `A10` et la chaîne vide.

# 4 Exercice 3 : Les erreurs

Complétez :

```
fn play(&mut self, row: usize, col: usize, player: Player)
    -> Result<(), MoveError>
```

La méthode doit :

1. Refuser toute position en dehors du plateau avec `Err(MoveError::InvalidPosition)`.
2. Refuser de remplacer une case occupée avec `Err(MoveError::Occupied)`.
3. Placer `Cell::Occupied(player)` dans une case vide puis retourner `Ok(())`.

Une tentative refusée ne doit **jamais** modifier le plateau. La boucle de jeu est déjà préparée pour afficher le message d'erreur et laisser le même joueur rejouer.

**Validation :** `play_rejects_occupied_cell` et `play_rejects_out_of_bounds` réussissent.

# 5 Exercice 4 : `Option`

L'ancienne fonction `check_victory() -> bool` ne disait pas **qui** avait gagné. Vous allez la remplacer par une information plus utile.

Complétez ces trois méthodes de `Board` :

```
fn matching_player(a: Cell, b: Cell, c: Cell) -> Option<Player>
fn winner(&self) -> Option<Player>
fn is_full(&self) -> bool
```

— `matching_player` retourne `Some(joueur)` si les trois cases appartiennent au même joueur, et `None` sinon. Vous pouvez utiliser `match (a, b, c)` et une garde `if`.

— `winner` examine les trois lignes, les trois colonnes et les deux diagonales ; il renvoie `Some(joueur)` dès qu'il détecte une victoire, sinon `None`.

— `is_full` retourne `true` si aucune case n'est `Cell::Empty`.

**Question :** Pourquoi `Option<Player>` est-il préférable à une valeur spéciale telle que `0` pour signifier « pas de gagnant » ?

**Validation :** `winner_detects_horizontal_vertical_and_diagonal` réussit.

# 6 Exercice 5 : Modéliser l'état de la partie

Le type `GameState` est fourni :

```
enum GameState {
    InProgress,
    Won(Player),
    Draw,
}
```

Complétez `TicTacToe::state(&self) -> GameState` :

— Si `Board::winner()` renvoie `Some(joueur)`, l'état est `GameState::Won(joueur)`.

— Sinon, si le plateau est plein, l'état est `GameState::Draw`.

— Sinon, l'état est `GameState::InProgress`.

**Important :** testez la victoire **avant** de tester le match nul. Une grille pleine peut contenir un alignement gagnant.

La méthode `launch()` est déjà écrite pour utiliser un `match` sur `GameState`. Observez les trois branches : partie en cours, victoire, match nul.

**Validation :** `state_reports_draw` et `state_prioritizes_victory_and_reset_works` réussissent.